

Three-Lens Prompt Card

Day 3 · Activity 3 handout. When you review another quad's C4 diagrams, read them through these three lenses. A reviewer who just nods isn't engaging — force a named scenario and walk it.

The three lenses

Lens	Question
Failure	Trace a failure: "What happens when the audit stream is full?" Walk the diagram aloud, arrow by arrow.
Trust boundary	Where does data cross from our control to someone else's? Mark it with a thick line.
Evolvability	If the architecture stance changes (modular monolith → microservice), which arrows would have to change?

Review protocol (with the diagram's author listening)

1. **Swap diagrams** (5 min). Read the other quad's Context and Container diagrams.
2. **Failure trace** (10 min). Pick one failure scenario; walk the diagram aloud. The author listens.
3. **Trust-boundary mark-up** (10 min). Mark where data crosses out of the author's control.
4. **Evolvability question** (10 min). Ask "if you were to split X into a new service, which arrows would change?"
5. **Author updates** (5 min). Authors annotate their diagram with the findings.

What good review surfaces

- **Trust boundaries are explicit** — diagrams without them hide compliance and security risk.
- **The failure trace exposes** a missing arrow or a missing failure-handling stance.
- **The evolvability conversation** produces 1–2 specific arrows to watch as the architecture matures.

Trust boundaries that follow the **team org chart** rather than the **data flow** are usually wrong. Challenge them. A "Database is part of our system" arrow, when the database is shared platform infra, is a trust-boundary error worth surfacing.

Capture (author fills in)

- **Failure scenario walked:**

- **Missing arrow / stance it exposed:**

- **Trust boundaries marked:**

- **Evolvability arrows to watch (1–2):**
